

TP N°5: Tipos Genéricos

Ejercicio 1

Implementar la clase `Pair<A,B>` que almacena dos objetos: uno de tipo A y otro de tipo B. El siguiente programa de prueba:

```
public class PairTester {  
  
    public static void main(String[] args) {  
        Pair<String, Double> stringDoublePair = new Pair<>("hola", 0.1);  
        System.out.println(stringDoublePair);  
        Pair<Integer, Integer> integerIntegerPair = new Pair<>(1, 2);  
        System.out.println(integerIntegerPair);  
        System.out.println(stringDoublePair.equals(new Pair<>("hola", 0.1)));  
    }  
}
```

produce la siguiente salida:

```
[hola, 0.1]  
[1, 2]  
true
```

¿El siguiente fragmento de código funciona? ¿Cuál es el tipo genérico inferido?

```
Pair otherPair = new Pair(1f, "mundo");  
System.out.println(otherPair);
```

Ejercicio 2

Se desea agregar a la implementación anterior la posibilidad de setear el contenido de las variables de instancia. Aprovechando la sobrecarga de métodos se propone la siguiente implementación:

```
public class Pair<A, B> {  
  
    private A a;  
    private B b;  
  
    ...  
  
    public void set(A a) {  
        this.a = a;  
    }  
  
    public void set(B b) {  
        this.b = b;  
    }  
}
```

¿Qué error se obtiene en compilación? ¿Cómo se puede solucionar?

Ejercicio 3

Parte 1

Modificar la implementación del **Ejercicio 12** del **TP N°4** para que tanto la implementación como la interfaz `LinkedList` sea genérica. Realizar el diagrama de clases correspondiente.

¿Qué ventaja tiene esta alternativa frente a la implementación anterior en donde se utiliza `Object`, si en definitiva en ambos casos se tiene una implementación genérica que permite almacenar cualquier tipo de elementos?

Parte 2

Modificar la implementación del **Ejercicio 4** del **TP N°4** para que las implementaciones del método `getPoints` en las clases `MovableRectangle`, `MovableTriangle`, `MovableEllipse` y `MovableCircle` **no requieran los casteos explícitos** a `MovablePoint`.

Tip: Implica modificar el tipo de dato de las variables de instancia que almacenan los `Point` en las figuras.

Ejercicio 4

Se cuenta con la clase `PrependLinkedListImpl<T>` que, a diferencia de la implementación anterior, agrega los elementos al principio de la lista.

¿Por qué la siguiente implementación falla en compilación? Corregirlo.

```
public class PrependLinkedListImpl<T> implements LinkedList<T> {

    private Node<T> first;

    @Override
    public void add(T obj) {
        first = new Node<>(obj);
    }

    ...

    private class Node<T> {

        private T head;
        private Node<T> tail;

        Node(T head) {
            this.head = head;
            this.tail = first;
            first = this;
        }

    }

}
```

Ejercicio 5

Diseñar una interfaz `Stack<E>` que modele una **pila de elementos genérica**. A continuación se muestra un fragmento de una implementación con arrays en la clase `ArrayStack<E>`. ¿Por qué falla en compilación? Corregirlo.

```
public class ArrayStack<E> implements Stack<E> {

    private E[] elements;
    private static final int INITIAL_DIM = 10;

    ...

    public ArrayStack() {
        elements = new E[INITIAL_DIM];
    }

    ...

}
```

Implementar todo lo necesario para que el siguiente programa de prueba

```
public class StackTester {

    public static void main(String[] args) {
        Stack<Integer> stack = new ArrayStack<>();
        stack.push(2);
        stack.push(3);
        System.out.println(stack);
        System.out.println(stack.peek());
        System.out.println(stack.pop());
        System.out.println(stack.isEmpty());
        System.out.println(stack.pop());
        System.out.println(stack.isEmpty());
        System.out.println(stack.pop());
    }

}
```

imprime la siguiente salida:

```
[3,2]
3
3
false
2
true
Exception in thread "main" java.util.EmptyStackException
    at ar.edu.itba.poo.tp5.stack.ArrayStack.pop(ArrayStack.java:34)
    at ar.edu.itba.poo.tp5.stack.StackTester.main(StackTester.java:15)
```

Realizar el diagrama de clases correspondiente.

Ejercicio 6

Implementar la clase `AccessStack`. Debe contar con dos métodos para conocer el número de veces que se invocó a los métodos *push* y *pop*. Además de la nueva implementación `AccessStack<E>` ¿corresponde crear una nueva interfaz? **Realizar el diagrama de clases correspondiente.**

Interfaces Comparable y Comparator

Ejercicio 7

Consultar la documentación de los métodos de clase `sort` y `toString` de la clase `Arrays`. ¿Cuál es la salida del siguiente programa de prueba? Justificar.

```
import java.util.Arrays;

public class ArrayTester {

    public static void main(String[] args) {
        Integer[] intArray = new Integer[]{7, 3, 1, 5, 9};
        System.out.println(Arrays.toString(intArray));
        Arrays.sort(intArray);
        System.out.println(Arrays.toString(intArray));
    }
}
```

Ejercicio 8

Indicar la salida del siguiente fragmento de código adicionado al programa de prueba del ejercicio anterior.

```
Pair<String, String> stringPair1 = new Pair<>("hola", "mundo");
Pair<String, String> stringPair2 = new Pair<>("hola", "adiós");
Pair<String, String> stringPair3 = new Pair<>("buen", "día");
Pair pairArray[] = new Pair[]{stringPair1, stringPair2, stringPair3};
Arrays.sort(pairArray);
System.out.println(Arrays.toString(pairArray));
```

Implementar todo lo necesario para que las instancias de la clase `Pair` puedan ordenarse en un array. A modo de ejemplo, para el fragmento anterior, se espera obtener la siguiente salida:

```
[[buen, día],[hola, adiós],[hola, mundo]]
```

donde el orden natural de los pares es ascendente por el primer campo y luego por el segundo.

Luego, indicar la salida del siguiente fragmento de código adicionado al programa de prueba del ejercicio anterior.

```
...
Pair<String, String>[] otherPairArray = new Pair[4];
otherPairArray[1] = stringPair1;
otherPairArray[2] = stringPair2;
otherPairArray[3] = stringPair3;
Arrays.sort(otherPairArray);
System.out.println(Arrays.toString(otherPairArray));
```

Ejercicio 9

Indicar la salida del siguiente fragmento de código adicionado al programa de prueba del ejercicio anterior.

```
Arrays.sort(intArray, new Comparator<Integer>() {
    @Override
    public int compare(Integer o1, Integer o2) {
        return o2.compareTo(o1);
    }
});
System.out.println(Arrays.toString(intArray));
```

¿Se puede simplificar el código del programa de prueba?

Ejercicio 10

Modificar el fragmento de código del **Ejercicio 8** para que el arreglo de instancias de **Pair** tengan el orden inverso al orden natural. A modo de ejemplo, con el nuevo orden, se espera obtener la siguiente salida:

```
[[hola, mundo],[hola, adiós],[buen, día]]
```

Ejercicio 11

Retomando la implementación del enumerativo **Continent** del **Ejercicio 14** del **TP N°4**, se cuenta con el siguiente de prueba:

```
public class ContinentTester {

    public static void main(String[] args) {
        Continent[] continents = new Continent[]{
            Continent.OCEANIA,
            Continent.AMERICA,
            Continent.EUROPE};
        Arrays.sort(continents);
        for(Continent continent : continents) {
            System.out.println(continent);
        }
    }
}
```

que produce la siguiente salida:

América
Europa
Oceanía

Consultar la documentación de la clase `java.lang.Enum` que utiliza Java para modelar a los enumerativos. ¿Qué criterio de ordenamiento se utiliza en el programa de prueba? ¿Es posible que el `enum Continent` implemente la interfaz `Comparable`? ¿Y que sobrescriba el método `compareTo`? ¿Por qué? ¿Puede sobrescribir los métodos `equals` y `hashCode`?

Hacer los cambios necesarios para que el arreglo de continentes tenga orden ascendente por población, de forma que la salida a obtener sea la siguiente:

Oceanía
Europa
América

Interfaces Iterable e Iterator

Ejercicio 12

Se desea contar con la clase `LocalDateInterval` para definir un intervalo de fechas (desde-hasta) con cierto paso definido en cantidad de días.

Implementar todo lo necesario para que, con el siguiente programa,

```
import java.time.LocalDate;
import java.util.Iterator;

public class LocalDateIntervalTester {

    public static void main(String[] args) {
        // Ejemplo de uso de la clase LocalDate
        LocalDate today = LocalDate.of(2020, 3, 1);
        System.out.println(today);
        LocalDate todayPlus10 = today.plusDays(10);
        System.out.println(todayPlus10);
        System.out.println(todayPlus10.isAfter(today));
        System.out.println("-----");
        // Ejemplo de uso de la clase LocalDateInterval
        LocalDateInterval localDateInterval = new LocalDateInterval(today,
todayPlus10, 3);
        for(LocalDate localDate : localDateInterval) {
            System.out.println(localDate);
        }
        System.out.println("-----");
        Iterator<LocalDate> localDateIterator = new LocalDateInterval(today,
today.plusDays(2), 1).iterator();
        while(localDateIterator.hasNext()) {
            System.out.println(localDateIterator.next());
        }
    }
}
```

se obtenga la siguiente salida:

```
2020-03-01
2020-03-11
true
-----
2020-03-01
2020-03-04
2020-03-07
2020-03-10
-----
2020-03-01
2020-03-02
2020-03-03
```

Ejercicio 13

Modificar la implementación del **Ejercicio 8** del **TP N°4** para que el intervalo sea iterable. Actualizar el diagrama de clases. Escribir un programa de prueba.

Ejercicio 14

Implementar la clase **ReversedIterator** que recibe un array y permite recorrerlo en sentido inverso. Para el siguiente programa de prueba:

```
public class ReversedIteratorTester {  
  
    public static void main(String[] args) {  
        String[] words = new String[]{"hola", "que", "tal"};  
        ReversedIterator<String> arrayReversedIterator = new  
ReversedIterator<>(words);  
        while(arrayReversedIterator.hasNext()) {  
            System.out.println(arrayReversedIterator.next());  
        }  
        try {  
            arrayReversedIterator.next();  
        } catch (Exception ex) {  
            System.out.println(ex.getClass());  
        }  
    }  
}
```

se debe obtener la siguiente salida:

```
tal  
que  
hola  
class java.util.NoSuchElementException
```

Ejercicio 15

Un **iterador cíclico** es un iterador que, a partir de una colección, permite recorrerla infinitamente. Al igual que un iterador convencional permite recorrer todos los elementos de la colección, pero la diferencia es que, una vez consumido todos los elementos de la colección, vuelve a recorrer los mismos elementos.

Se desea implementar un iterador cíclico que recibe un entero **N** en su constructor y, ante cada invocación a **next**, retorne **N elementos de la colección**.

El método **next** lanza una **NoSuchElementException** cuando una invocación a **hasNext** hubiera retornado **false**.

Implementar todo lo necesario para que, con el siguiente programa:

```
import java.util.Arrays;  
  
public class CyclicWindowIteratorTester {  
  
    public static void main(String[] args) {  
        String[] words = new String[]{"hola", "que", "tal"};
```



```
CyclicWindowIterator<String> iterator = new CyclicWindowIterator<>(words, 2);
for(int i = 0; i < 5; i++) {
    System.out.println(Arrays.toString(iterator.next()));
}
Integer[] empty = new Integer[0];
CyclicWindowIterator<Integer> emptyIterator = new
CyclicWindowIterator<>(empty, 2);
System.out.println(emptyIterator.hasNext());
try {
    emptyIterator.next();
} catch (Exception ex) {
    System.out.println(ex.getClass());
}
}
```

se obtenga la siguiente salida:

```
[hola, que]
[que, tal]
[tal, hola]
[hola, que]
[que, tal]
false
class java.util.NoSuchElementException
```